🧪 Autonomous AI Lab Assistant Using Deep Learning

Complete Architecture, Step-by-Step Manual & Project Viva Guide

GitHub Repository: https://github.com/Sreehasa123/autonomous-ai-lab-assistant-dl

---

1. Executive Summary & Main Objective

In academic institutions, students frequently struggle when conducting Deep Learning (DL) laboratory experiments. While high-level concepts are taught in lectures, hands-on implementation presents steep barriers:

- **Dataset Ambiguity:** Students don't know which dataset suits their problem statement or what realistic accuracy to expect.
- **Data Leakage Traps:** Fitting scalers/imputers before train/test splitting or augmenting test sets.
- **Catastrophic PyTorch Mismatches:** Placing `nn.Softmax()` before `nn.CrossEntropyLoss()`, causing double-softmaxed, severely flattened gradients.
- **Training Instabilities:** Exploding gradients (`Loss is NaN`), missing `optimizer.zero_grad()`, or improper learning rates.
- **Accuracy Paradox:** Believing a 95% accuracy model on an imbalanced dataset is successful when it merely predicts the majority class.
- **Lab Report Writing Overhead:** Spending hours manually drafting and formatting IEEE-style reports with LaTeX equations.

> ***Main Objective:*** *To develop an autonomous, pedagogical, RAG-guided multi-agent AI assistant that coaches students step-by-step through their deep learning experiments—from dataset discovery with expected accuracy targets to data preprocessing, architecture design, code synthesis, real-time error debugging, experiment history storage, and publication-ready academic report generation.*

2. What Makes This App Different (USP & Comparisons)

Dimension	Generic LLMs (ChatGPT / Copilot)	AutoML (AutoKeras, Cloud AutoML)	Autonomous AI Lab Assistant (Our App)
Educational Goal	Answers questions in isolation without structured state.	"Black box" engine hiding code; student learns nothing.	**Pedagogical Co-Pilot:** Explains *why*, warns of traps, and generates clean PyTorch code.
Domain-Specific RAG	Relies purely on generic weights; hallucinates subtle ML traps.	None; only searches hyperparameters.	**Curated DL Rulebook:** 12+ verified domain playbooks covering leakage, Softmax bugs, and gradient clipping.
Dataset Benchmarks	Names datasets without realistic accuracy targets.	Requires user to supply ready data.	**Benchmarked Suggestions:** Suggests datasets from `torchvision`/Hugging Face with achievable student accuracy ranges.
Evaluation Intelligence	Usually recommends just accuracy and loss.	Reports standard metrics.	**Context-Aware Metric Suite:** Categorizes Primary vs Secondary diagnostics (Macro-F1, PR-AUC, Confusion Matrix).
Experiment Persistence	Chat is ephemeral; hard to iterate.	Opaque cloud models.	**JSON State Store:** Save, reload, and branch experiments to iterate at any time.
Lab Report Synthesis	Must manually prompt and piece together text.	Exports raw CSV numbers.	**Automated IEEE Report:** Formulates LaTeX equations, tables, error analyses, and markdown export.

3. Technology Stack Breakdown

- Streamlit 1.32+ **Frontend UI:** Python-native reactive dashboard with tabs, dynamic metric cards, and clean forms.
- Google Gemini 3.8 Flash **Generative Engine:** Low-latency LLM with 1M token context for dataset reasoning, code generation, error diagnosis, and report writing.
- RAG Architecture **Grounding Engine:** In-memory domain knowledge base preventing hallucination and enabling offline advice.
- PyTorch & Torchvision **Deep Learning Core:** Modular `nn.Module` architectures, transfer learning (ResNet-18), and data augmentations.
- Scikit-Learn & Pandas **Data & Metrics:** Stratified splits, scaling, confusion matrices, and classification metrics.
- JSON Persistence **Local Storage:** Lightweight local file database in `workspace/experiments_history.json`.

4. The 5 Top Navigation Tabs & Their Metrics

🧭 Tab 1: Step-by-Step Lab Guide

Guides students through all 6 experimental phases sequentially with instant advice and quick-prompt buttons.

Metrics Tracked: Benchmark accuracy ranges, class imbalance ratio, missing value ratio, 80/10/10 split ratio, and training loss progression.

💻 Tab 2: Starter Code Generator

Automatically generates a clean, modular, end-to-end Python script matching the student's exact choices, ready to download as `experiment.py`.

Enforced Metrics & Safeguards: Logits integrity (no redundant Softmax), gradient clipping bound (max_norm=1.0), device placement consistency, and train/eval phase switching.

🛠️ Tab 3: Code & Error Debugger (AI Problem Solver)

Students paste broken code and error tracebacks. The assistant provides: 1. Root Cause Analysis, 2. DL Concept Explanation, 3. Corrected Replacement Code, and 4. Prevention Tips.

Diagnostic Metrics Handled: Tensor dimension mismatches (matrix multiplication compatibility), exploding gradients (NaN loss), CUDA Out of Memory (OOM), and missing `zero_grad()`.

📝 Tab 4: Automated Lab Report Generator

Compiles all inputs, parameters, and observations into an IEEE-standard academic laboratory report with one-click Markdown (.md) download.

Report Metrics: 9 mandatory sections, LaTeX mathematical equations for loss/activations, and hyperparameter tables.

📚 Tab 5: Knowledge Base Rules Explorer

Searchable library of 12+ verified deep learning playbooks covering pitfalls, recommended practices, and verified PyTorch code patterns.

5. The 6 Deep Learning Lab Steps Explained

1. **Step 1: Dataset & Problem Setup:** Analyzes problem title and aim, recommends datasets with expected student accuracy benchmarks, and auto-populates the workspace.
2. **Step 2: Preprocessing & Leakage Checker:** Verifies scaling is fitted only on training data (prevents data leakage) and checks augmentations.
3. **Step 3: Model Architecture & Loss:** Recommends appropriate backbones and verifies logits are passed directly to CrossEntropyLoss.

4. **Step 4: Training & Hyperparameter Tuning:** Recommends gradient clipping, AdamW, early stopping, and schedulers.
5. **Step 5: Evaluation & Error Analysis:** Recommends task-specific metrics (Macro-F1, Confusion Matrix) and diagnoses overfitting vs underfitting from loss curves.
6. **Step 6: Lab Report Generator:** Consolidates the complete experiment into an academic report.

6. Comprehensive Metric Guide

Metric	Formula	Why It Matters in Deep Learning Labs
Accuracy	(TP + TN) / Total	Overall benchmark for balanced datasets. Misleading when classes are skewed.
Validation Loss	-∑ y_i log(ŷ_i)	Monitors generalization during training. Rising validation loss signals overfitting.
Macro-F1 Score	Average of F1 across all classes	Essential for imbalanced datasets; ensures minority classes aren't ignored.
Precision	TP / (TP + FP)	Penalizes False Positives (vital in spam detection, fraud detection).
Recall / Sensitivity	TP / (TP + FN)	Penalizes False Negatives (vital in medical diagnosis / disease screening).
Confusion Matrix	2D Contingency Table	Identifies which exact pairs of classes the model confuses.
PR-AUC	Area under PR curve	The most informative metric when positive cases make up <5% of samples.

7. Quickstart Guide (How to Run)

```
# 1. Clone repository git clone https://github.com/Sreehasa123/autonomous-ai-lab-assistant-dl.git cd autonomous-ai-lab-assistant-dl # 2. Set up virtual environment python
```

```
-m venv venv .\venv\Scripts\activate # Windows # 3. Install dependencies pip install -r requirements.txt # 4. Run application streamlit run app.py
```

8. Project Viva / Presentation Cheat Sheet

Q: What is the main objective of this project?

A: To develop an autonomous, pedagogical, RAG-guided multi-agent AI assistant that coaches students step-by-step through their deep learning experiments—from dataset discovery with expected accuracy targets to data preprocessing, architecture design, code synthesis, real-time error debugging, experiment history storage, and publication-ready academic report generation.

Q: Why use RAG instead of standard ChatGPT?

A: Standard LLMs hallucinate subtle deep learning traps (like double Softmax) and provide disconnected answers without lab context. Our RAG engine grounds responses in verified deep learning rules and works even in offline mode.

Q: What are the main features on the top navigation bar?

A: 1. Step-by-Step Lab Guide, 2. Starter Code Generator, 3. Code & Error Debugger (AI Problem Solver), 4. Automated Lab Report Generator, and 5. Knowledge Base Rules Explorer.
